Day-18 Python 90 Days Challenge

10.5 Arguments

An **argument** is the actual value passed to a function during function call.

Syntax

```
function_name(argument)
```

Explanation

1. Parameter is written in function definition.
2. Argument is passed during function call.
3. Arguments provide real values to parameters.

Parameter vs Argument

Term	Where It Appears	Meaning
Parameter	Function definition	Variable that receives value
Argument	Function call	Actual value passed

Example 1

PYTHON CODE

```
def greet(name):
    print("Hello", name)
greet("Riya")
```

Output: Hello Riya

Code	Role
name	Parameter
"Riya"	Argument

10.6 Positional Arguments

Positional arguments are matched based on their position.

Syntax

```
function_name(argument1, argument2)
```

Codewith Nischal

Explanation

1. Python passes arguments in the same order as parameters.
2. The first argument goes to the first parameter.
3. The second argument goes to the second parameter.
4. Order matters in positional arguments.

Example 1

```
PYTHON CODE
def student_info(name, age):
    print("Name:", name)
    print("Age:", age)

student_info("Aman", 21)
```

Output:
Name: Aman
Age: 21

Flow Chart

student_info("Aman", 21)

| |

v v

name age

10.7 Return Values

A **return value** is the result sent back by a function.

Syntax

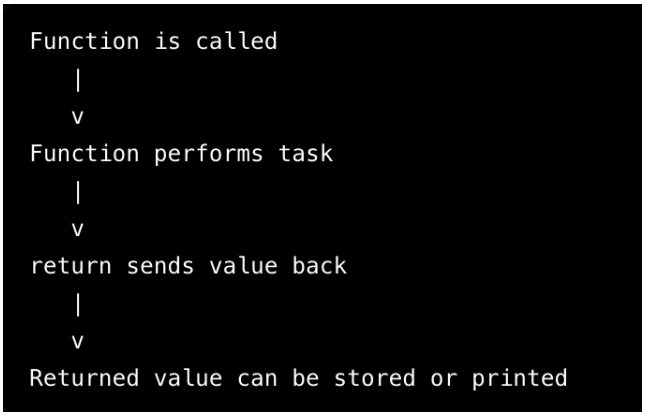
def function_name():

return value

1. return sends a value back to the place where the function was called.
2. A function can return one value or multiple values.
3. After return, the function stops executing.
4. If there is no return, Python returns None automatically.

CodeWithNishchal

Flow Chart



Example 1

PYTHON CODE

```
def add(a, b):
    result = a + b
    return result

answer = add(10, 20)
print(answer)
```

Output:

30

Example 2: Function without return

PYTHON CODE

```
def greet():
    print("Hello")

result = greet()
print(result)
```

Output: Hello
None

Because greet() does not return any value, Python returns None.

deWithNishchal

10.8 Returning Multiple Values

Python functions can return multiple values.

Syntax:

```python
def function_name():
    return value1, value2
```

1. Multiple values can be returned using commas.
2. Python returns them as a tuple.
3. Returned values can be unpacked into variables.

Example 1

PYTHON CODE

```
def calculate(a, b):
    total = a + b
    difference = a - b
    return total, difference
sum_result, diff_result = calculate(20, 10)
print(sum_result)
print(diff_result)
```

Output:

30
10

10.9 Default Parameters

A default parameter has a predefined value. If no argument is passed, the default value is used.

Syntax

```python
def function_name(parameter=value):
    statement
```

1. Default parameters make arguments optional.
2. If an argument is provided, Python uses the given value.
3. If no argument is provided, Python uses the default value.
4. Non-default parameters must come before default parameters.

CodeWithNishchal

Example 1

PYTHON CODE

```
def greet(name="Guest"):
    print("Hello", name)
greet("Aman")
greet()
```

Output:

Hello Aman

Hello Guest

Correct and Incorrect Order

Code	Valid?	Reason
def show(name, age=18):	Yes	Default parameter comes after normal parameter
def show(name="Guest", age=18):	Yes	Both have defaults
def show(name="Guest", age):	No	Non-default parameter cannot come after default parameter

Important Warning: Avoid Mutable Default Values

Do not use mutable objects like list or dictionary as default values.

Wrong style:

```
def add_item(item, items=[]):
    items.append(item)
    return items

print(add_item("A"))
print(add_item("B"))
```

Output: ['A']

['A', 'B']

The same list is reused between function calls.

Better style:

Python

```
def add_item(item, items=None):
    if items is None:
        items = []

    items.append(item)
    return items

print(add_item("A"))
print(add_item("B"))
```

Output: ['A']

['B']

Coded with Kushal

10.10 Keyword Arguments

Keyword arguments pass values using parameter names.

Syntax

```
function_name(parameter_name=value)
```

1. Keyword arguments use names while calling a function.
2. Order does not matter when keyword arguments are used.
3. They make function calls more readable.
4. Positional arguments must come before keyword arguments.

Example 1

PYTHON CODE

```
def student_info(name, age, course):
    print("Name:", name)
    print("Age:", age)
    print("Course:", course)
student_info(age=21, course="Python", name="Aman")
```

Output: Name: Aman
Age: 21
Course: Python

Positional vs Keyword Arguments

Type	Example	Order Matters?
Positional argument	student_info("Aman", 21, "Python")	Yes
Keyword argument	student_info(age=21, name="Aman", course="Python")	No

Important Rule

Correct:

```
student_info("Aman", age=21, course="Python")
```

Incorrect:

```
student_info(name="Aman", 21, "Python")
```

Positional arguments cannot come after keyword arguments.

CodeWithNishchal

10.11 *args

*args is used when we do not know how many positional arguments will be passed.

Syntax

```
def function_name(*args):
    statement
```

1. *args collects extra positional arguments.
2. The collected values are stored as a tuple.
3. The name args is a convention; the * is important.
4. Use *args when the number of arguments is flexible.

Flow Chart

Function call has many positional arguments
  |
  v
*args collects them
  |
  v
Values are stored as a tuple

Example 1

PYTHON CODE

```
def add_numbers(*numbers):
    total = 0
    for number in numbers:
        total += number
    return total
print(add_numbers(10, 20, 30))
print(add_numbers(5, 15))
```

Output:
60
20
Here, numbers behaves like a tuple.

WithNishchal

10.12 **kwargs

**kwargs is used when we do not know how many keyword arguments will be passed.

Syntax

```
def function_name(**kwargs):
    statement
```

Explanation

1. **kwargs collects extra keyword arguments.
2. The collected values are stored as a dictionary.
3. Keys are argument names.
4. Values are argument values.
5. The name kwargs is a convention; the ** is important.

Flow Chart

Function call has many keyword arguments
  |
  v
**kwargs collects them
  |
  v
Values are stored as a dictionary

Example 1

PYTHON CODE

```
def show_profile(**details):
    for key, value in details.items():
        print(key, value)


show_profile(name="Aman", age=21,
course="Python")
```

Output:
name Aman
age 21
course Python

Here, details behaves like a dictionary.

deWithNishchal

10.13 *args vs **kwargs

Point	*args	**kwargs
Collects	Positional arguments	Keyword arguments
Stored as	Tuple	Dictionary
Symbol	Single star *	Double star **
Example call	func(10, 20, 30)	func(name="Aman", age=21)

Example 1

PYTHON CODE
```
def show_data(*args, **kwargs):
    print(args)
    print(kwargs)

show_data(10, 20, name="Aman", age=21)
```

Output:
(10, 20)
{'name': 'Aman', 'age': 21}

10.14 Function Parameter Order

When using different types of parameters together, the order matters.

Syntax

```
def function_name(normal_parameter, default_parameter=value, *args, **kwargs):
    statement
```

Explanation

1. Normal parameters come first.
2. Default parameters come after normal parameters.
3. *args comes after normal/default parameters.
4. **kwargs comes last.
5. This order keeps function calls clear and valid.

CodeWithNishchal

Parameter Order Table

Order	Parameter Type	Example
1	Normal parameter	name
2	Default parameter	age=18
3	*args	*marks
4	**kwargs	**details

Example 1

```
def student_report(name, age=18, *marks, **details):
    print("Name:", name)
    print("Age:", age)
    print("Marks:", marks)
    print("Details:", details)

student_report("Aman", 21, 80, 90, 85, course="Python", city="Delhi")
```

Output:

Name: Aman
Age: 21
Marks: (80, 90, 85)
Details: {'course': 'Python', 'city': 'Delhi'}

10.15 Docstrings

A **docstring** is a string written inside a function to explain what the function does.

Syntax

```
def function_name():
    """Function description."""
    statement
```

1. A docstring is written using triple quotes.
2. It is usually written as the first statement inside a function.
3. It explains the purpose of the function.
4. It helps other programmers understand the function.
5. It can be viewed using help() or .__doc__.

CodeWithNishchal

Example 1

```
def add(a, b):
    """Return the sum of two numbers."""
    return a + b

print(add(10, 20))
print(add.__doc__)
```

Output: 30

Return the sum of two numbers.

Good Docstring Style

```
Python

def calculate_area(length, width):
    """
    Return the area of a rectangle.

    length: rectangle length
    width: rectangle width
    """
    return length * width
```

10.16 Function Naming Rules

Function names follow the same basic rules as variable names.

Syntax

```
def function_name():
    statement
```

1. Function names should be meaningful.
2. Use snake_case for function names.
3. Function names should usually describe an action.
4. Avoid using Python keywords as function names.
5. Avoid unclear names like x(), abc(), or test1().

CodeWithNishchal